CHAPTER 5 — SOFTWARE DESIGN

Doctor Vocabulary Coverage Ledger

Module 1: Software Design Fundamentals

Term 01: What is Design?

Canonical Definition:

Software design is the iterative engineering process through which software requirements specifications (SRS) are transformed into a technical blueprint for construction. It specifies the structural architecture, component interfaces, data structures, and procedural algorithms required to build the software system.

Architectural Mechanism:

- Serves as the pivotal bridge in the SDLC transitioning from the problem domain (requirements analysis answering *what*) into the solution domain (implementation answering *how*).
- Decomposes monolithic system behavior into intellectually manageable, verifiable modular components.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 203–204; Companion Slides 2–4.

Term 02: Conceptual and Technical Designs

Canonical Definition:

Conceptual Design (External Design) models the system from the customer and user perspective, focusing on "WHAT" capabilities are delivered; Technical Design (Internal Design) models the system from the builder perspective, focusing on "HOW" the architecture executes.

Architectural Mechanism:

- **Conceptual Design:** Produces UI mockups, screen layouts, workflows, business operations, and external data formats in non-technical domain language.
- **Technical Design:** Produces module hierarchies, interface signatures, database schemas, and pseudo-code in formal engineering specifications (IEEE Std 1016).
- **Architect Role:** Software designers serve as the intellectual bridge translating conceptual client expectations into technical builder blueprints.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 204–205; Companion Slides 4–5, Fig. 2.

Term 03: Objectives of Design

Canonical Definition:

The design objectives are: correct and complete; understandable; at the right level; and maintainable, while facilitating maintenance.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §5.1.2, printed pp. 205–206.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - الأهداف الأربعة هي نفس ألفاظ المصدر، لا تستبدل "at the right level" بقائمة صفات إضافية.

Term 04: Why Design is Important?

Canonical Definition:

Software design turns requirements into a plan for constructing the software and influences the quality and maintainability of the result. Vitruvius's three qualities are **durability, utility, and charm**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 203–206.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - احذف أرقام كلفة الصيانة أو مضاعفات كلفة إصلاح العيوب ما لم يسندها مصدر محدد لهذا الادعاء.

Module 2: Modularity, Coupling & Cohesion

Term 05: Modularity

Canonical Definition:

Modularity is the software architectural attribute whereby a system is decomposed into discrete, logically cohesive, and loosely coupled units (modules) interacting through defined interfaces.

Architectural Mechanism & Bertrand Meyer's Five Criteria:

To achieve robust quality, modular decomposition must satisfy Bertrand Meyer's five criteria (Pressman 5e §13.4.3):

1. **Modular Decomposability:** Systematic division of complex problems into smaller sub-problems.
2. **Modular Composability:** Recombining self-contained modules to build new systems.
3. **Modular Understandability:** Comprehending a module in isolation without reading other modules.
4. **Modular Continuity:** Minor requirements changes cause changes in only a single localized module.
5. **Modular Protection:** Runtime faults and exceptions are confined locally within the originating module.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 206–207; Companion Slides 14–15; Pressman §13.4.3.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- النمطية (Modularity) تخضع لمعايير ماير الخمسة: Decomposability ← Composability ← Understandability ← Continuity ← Protection. تذكر أن النمطية الاقتصادية المثلى تقف عند النقطة M في منحنى التكلفة.

Term 06: Module Coupling

Canonical Definition:

Module coupling is the qualitative measure of the degree of interdependence and structural connectivity between two software modules. Software engineering strictly strives for *loose coupling*.

The Six Levels of Coupling (Ranked Best to Worst):

1. **Data Coupling (Loosest / Most Desirable):** Modules communicate strictly through atomic parameter values (e.g., integers, floats).
2. **Stamp Coupling:** Modules communicate by passing composite data structures, but the called module uses only a subset of fields.
3. **Control Coupling:** One module passes a control flag or opcode intended to dictate internal branching in another.
4. **External Coupling:** Modules bind to external protocols, formats, or hardware device environments.
5. **Common Coupling:** Modules read and write to unencapsulated shared global variables or common memory blocks.
6. **Content Coupling (Tightest / Worst):** One module directly branches into or mutates local variables and code inside another.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 207–210; Companion Slides 15–24.

Term 07: Module Cohesion

Canonical Definition:

Module cohesion is the qualitative measure of the internal functional strength and degree of singularity of purpose among elements within a single module. Software engineering strictly strives for *high cohesion*.

The Seven Levels of Cohesion (Ranked Worst to Best):

1. **Coincidental Cohesion (Weakest / Worst):** Elements are bundled arbitrarily with no meaningful relationship.
2. **Logical Cohesion:** Elements perform a class of logically related activities, selected by a control flag at runtime.
3. **Temporal Cohesion:** Elements execute together during the same timeframe (e.g., startup initialization).
4. **Procedural Cohesion:** Elements execute in a specific sequential order, but operate on different data.
5. **Communicational Cohesion:** Elements perform different operations, but operate on the exact same input/output data.
6. **Sequential Cohesion:** The output of one operation forms the direct input to the next operation in an assembly line.
7. **Functional Cohesion (Strongest / Most Desirable):** Every instruction cooperates exclusively to perform a single, well-defined functional task.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 210–213; Companion Slides 25–30.

Term 08: Relationship between Cohesion & Coupling

Canonical Definition:

A good modular design aims for high cohesion within modules and low coupling between modules. These are complementary design goals; high cohesion does not by itself guarantee low coupling.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 212–213; Companion Slide 31.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - احفظهما هدفين متلازمين للتصميم، ولا تصغ العلاقة كقانون سببي مضمون.

Module 3: Strategy of Design

Term 09: Strategy of Design

Canonical Definition:

A design strategy is the overarching methodological direction and approach adopted to guide the decomposition and synthesis of a software system from requirements into architecture.

Architectural Mechanism:

- Determines whether engineering activities proceed top-down (decomposition of system goals), bottom-up (composition of primitive assets), or via a balanced hybrid synthesis.
- Governs the sequencing of module specification, interface stability, and early prototyping risks.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–215; Companion Slide 32.

Term 10: Bottom-Up Design

Canonical Definition:

Bottom-Up Design is the strategy that begins by designing and implementing the lowest-level basic components, utilities, and primitives, progressively combining them upward into higher-level subsystems.

Architectural Mechanism:

- Highly effective for building reusable utility libraries, hardware drivers, and foundational data structures.
- *Limitation:* Lacks early global architectural guidance; risks constructing subsystems that fail to align with high-level user requirements. Requires test drivers to invoke lower modules.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–214; Companion Slide 33.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- التصميم التصاعدي (Bottom-Up): يبدأ من الوحدات الخدمية الصغرى صعوداً. ممتاز لإعادة الاستخدام، لكنه ضعيف في التركيز على أهداف النظام الكلية المبكرة.

Term 11: Top-Down Design

Canonical Definition:

Top-Down Design is the strategy that begins at the highest system level (executive coordinator) and iteratively decomposes functionality into progressively lower-level sub-modules through stepwise refinement.

Architectural Mechanism:

- Keeps system objectives, business workflows, and global interfaces in sharp focus throughout development.
- *Limitation:* Postpones low-level hardware and library integration. Requires extensive test stubs to simulate unimplemented subordinate modules during early testing.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 214–215; Companion Slide 34.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- التصميم التنازلي (Top-Down): يبدأ من رأس الهرم بالتفكيك التدريجي. يحافظ على أهداف النظام، لكنه يحتاج إلى بدائل وهمية (Stubs) لاختبار الوحدات العليا قبل اكتمال الوحدات السفلى.

Term 12: Hybrid Design

Canonical Definition:

Hybrid design combines top-down and bottom-up strategies. It can use top-down decomposition for system goals while incorporating lower-level components where suitable.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 215; Companion Slide 35.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - المصدر يشرح الجمع بين الاستراتيجيتين؛ لا تصفه كمعيار صناعي مهيمن أو كضمان لتقليل الوقت والأخطاء.

Module 4: Function-Oriented Design (FOD)

Term 13: Function Oriented Design

Canonical Definition:

Function-Oriented Design (FOD / Structured Design) is an architectural approach where a software system is decomposed into a structured hierarchy of procedural functions and subroutines operating upon passing data streams.

Architectural Mechanism:

- Views software as a computational process transforming inputs into outputs.
- Separates data structures from the procedural algorithms that manipulate them, modeling the system primarily through Data Flow Diagrams (DFDs) and Structure Charts.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216–220; Companion Slides 36–40.

Term 14: Design Notations

Canonical Definition:

Design notations are formal graphical and textual representational systems used to model different architectural dimensions (data flow, control hierarchy, dynamic behavior, and physical deployment) of a software system.

Architectural Mechanism:

- **Data Flow Diagrams (DFDs):** Model data movement, stores, and transformations without control flow or sequence.
- **Structure Charts:** Model procedural control hierarchies, module invocation, conditional calls, and data/control couples.
- **UML Diagrams:** Standard multi-view notations (Class, Sequence, State, Component) modeling object-oriented systems.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216–221; Companion Slides 41–46.

Term 15: Functional Procedure Layers

Canonical Definition:

Functional Procedure Layers are four levels of detail used to specify a procedure:

1. **Level 0:** Name, purpose, system relationships, callers, and author/date.
2. **Level 1:** Parameters, global variables, called routines, side effects, and input/output assertions.
3. **Level 2:** Local data structures, timing constraints, exception handling, and limitations.
4. **Level 3:** Procedure body using a structured chart, pseudocode, decision table, flowchart, or similar notation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 220–221; Companion Slides 47–49.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - هي مستويات تفصيل لوصف الإجراء من 0 إلى 3، وليست ثلاث طبقات معمارية.

Module 5: IEEE Recommended Practice for Software Design Descriptions

Term 16: IEEE Recommended Practice for Software Design Descriptions

Canonical Definition:

IEEE Std 1016-1998 is the standard recommended practice defining the information content, structure, and organization of a Software Design Description (SDD).

Architectural Mechanism:

- Establishes an industry-standard technical blueprint that communicates design decisions from architects to developers, testers, and maintainers across the software lifecycle.
- *Lineage Invariant:* Course scope strictly adheres to the 1998 revision.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 221–224; Companion Slide 50; IEEE Std 1016-1998.

Term 17: Scope

Canonical Definition:

The Scope of IEEE Std 1016 defines the applicability of the standard to the description of software designs, specifying the required content and organization of an SDD without mandating a specific design methodology.

Architectural Mechanism:

- Applies universally to commercial, scientific, embedded, and military software.
- Independent of whether function-oriented or object-oriented methodologies are employed.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 221; Companion Slide 50; IEEE Std 1016 §1.

Term 18: Definitions

Canonical Definition:

The Definitions section of IEEE Std 1016 establishes formal terminology, defining a Design Entity as a discrete structural component, a Design View as an attribute subset addressing specific concerns, and an SDD as a technical representation of system architecture.

Architectural Mechanism:

- Unifies vocabulary across engineering teams, eliminating semantic ambiguity during formal architecture inspections.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 221–222; Companion Slide 51; IEEE Std 1016 §3.

Term 19: Purpose of an SDD

Canonical Definition:

The purpose of an SDD is to provide an authoritative technical specification that guides software implementation, establishes a baseline for verification and validation against the SRS, and provides lifecycle documentation for maintenance.

Architectural Mechanism:

- Eliminates architectural guessing by developers.
- Acts as the primary artifact inspected during formal design reviews and integration test planning.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 222; Companion Slide 52; IEEE Std 1016 §4.

Term 20: Design Description Information Content

Canonical Definition:

Information content in IEEE Std 1016 is structured around ten mandatory entity attributes: Identification, Type, Purpose, Function, Subordinates, Dependencies, Interfaces, Resources, Processing, and Data.

Architectural Mechanism:

- Every structural entity (module, class, subsystem) must specify these 10 attributes, ensuring complete structural, behavioral, and interface transparency.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 222–223; Companion Slides 52–53; IEEE Std 1016 §5.

Term 21: Design Description Organisation

Canonical Definition:

Design Description Organisation arranges the ten entity attributes into four standard Design Views: Decomposition View, Dependency View, Interface View, and Detailed Design View.

Architectural Mechanism:

- **Decomposition View:** Subsystem and module hierarchy.
- **Dependency View:** Inter-module coupling and external resource requirements.
- **Interface View:** Public method signatures, parameters, and communication contracts.
- **Detailed Design View:** Internal algorithms, state variables, and data structures for coding.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 223–225; Companion Slides 54–57; IEEE Std 1016 §6.

Module 6: Object-Oriented Design (OOD)

Term 22: Object-Oriented Design

Canonical Definition:

Object-oriented design organizes a software design around interacting objects and classes. The chapter's discussion includes abstraction, inheritance, and polymorphism as object-oriented concepts.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 225–230; Companion Slides 59–71.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - لا تنسب Open/Closed Principle إلى هذا التعريف ما لم يرد بسند مستقل واضح.

Term 23: Basic Concepts

Canonical Definition:

The foundational concepts of OOD comprise Objects (stateful instances), Classes (abstract templates), Messages (communication protocols), Abstraction (essential boundary), Inheritance (hierarchical reuse), and Polymorphism (dynamic dispatch).

Architectural Mechanism:

- **Inheritance:** Models "is-a" generalizations.
- **Polymorphism:** Enables code to interact with abstract base classes while runtime dispatch executes specialized derived methods.
- **Encapsulation:** Hides internal data representations behind public method contracts.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 226–230; Companion Slides 59–71.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- المفاهيم الكائنية الأساسية: الكائن، الصنف، الرسائل، التجريد، الوراثة، وتعدد الأشكال. الفخ: الصنف هو ADT مضافاً إليه الوراثة وتعدد الأشكال.

Term 24: Steps to Analyze and Design an Object-Oriented System

Canonical Definition:

The described sequence is:

1. Create a use-case model (identify actors, write use cases, draw the use-case diagram).
2. Create an activity diagram, **if required**.
3. Create interaction diagrams (identify objects for each use case; prepare sequence/collaboration diagrams per use case).
4. Create a class diagram.
5. Create state-chart and object diagrams, **if required**.
6. Create component and deployment diagrams.
7. Prepare design documents.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §5.6.2, printed pp. 230–233; Companion Slides 72–75.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - احتفظ بالخطوات الشرطية كما هي، ولا تستبدلها بمسار عام من كتاب آخر.

Topic-to-Question Map

Syllabus Module	Terms Covered	Primary Course Sources	Key Concepts & Frameworks
Module 1: Software Design	Term 01, 02, 03, 04	K.K. Aggarwal pp. 203–206; Slides 2–10	What is design, Conceptual vs Technical design, design objectives, importance & economics.
Module 2: Modularity	Term 05, 06, 07, 08	K.K. Aggarwal pp. 206–213; Slides 14–31	Modularity, Meyer's 5 criteria, 6 coupling levels, 7 cohesion levels, cohesion-coupling duality.
Module 3: Strategy of Design	Term 09, 10, 11, 12	K.K. Aggarwal pp. 213–215; Slides 32–35	Strategy definition, Bottom-Up, Top-Down, Hybrid design dominance.
Module 4: FOD	Term 13, 14, 15	K.K. Aggarwal pp. 216–221; Slides 36–49	Function-Oriented Design, multi-view design notations, functional procedure layers.
Module 5: IEEE Std 1016	Term 16, 17, 18, 19, 20, 21	K.K. Aggarwal pp. 221–225; Slides 50–57	IEEE 1016-1998 scope, definitions, purpose, 10 entity attributes, 4 design views.
Module 6: OOD	Term 22, 23, 24	K.K. Aggarwal pp. 225–233; Slides 58–75	OOD paradigm, basic concepts (ADT vs Class, inheritance, polymorphism), 6-step OOAD process.